

**40009 32**

## Haskell Cryptography

### Submitters

---

kw1425

# Emarking

```
1: Final Tests: Summary for kw1425 of c1
2: PPT 21
3: -----
4:
5:   Public Tests:
6:     student-tests/crypto-test/crypto/part 1/gcd:           8 / 8
7:     student-tests/crypto-test/crypto/part 1/phi:           8 / 8
8:     student-tests/crypto-test/crypto/part 1/modPow:        11 / 11
9:     student-tests/crypto-test/crypto/part 1/computeCoeffs:  6 / 6
10:    student-tests/crypto-test/crypto/part 1/inverse:        7 / 7
11:    student-tests/crypto-test/crypto/part 1/smallestCoPrimeOf: 6 / 6
12:    student-tests/crypto-test/crypto/part 1/genKeys:         6 / 6
13:    student-tests/crypto-test/crypto/part 1/rsaEncrypt:       4 / 4
14:    student-tests/crypto-test/crypto/part 1/rsaDecrypt:       4 / 4
15:    student-tests/crypto-test/crypto/part 2/alphaOrd:         3 / 3
16:    student-tests/crypto-test/crypto/part 2/alphaChr:         3 / 3
17:    student-tests/crypto-test/crypto/part 2/add:              3 / 3
18:    student-tests/crypto-test/crypto/part 2/subtract:         3 / 3
19:    student-tests/crypto-test/crypto/part 2/ecbEncrypt:       4 / 4
20:    student-tests/crypto-test/crypto/part 2/ecbDecrypt:       4 / 4
21:    student-tests/crypto-test/crypto/part 2/cbcEncrypt:       4 / 4
22:    student-tests/crypto-test/crypto/part 2/cbcDecrypt:       4 / 4
23:    original-tests/crypto-test/crypto/part 1/gcd:            8 / 8
24:    original-tests/crypto-test/crypto/part 1/phi:             8 / 8
25:    original-tests/crypto-test/crypto/part 1/modPow:          11 / 11
26:    original-tests/crypto-test/crypto/part 1/computeCoeffs:   6 / 6
27:    original-tests/crypto-test/crypto/part 1/inverse:         7 / 7
28:    original-tests/crypto-test/crypto/part 1/smallestCoPrimeOf: 6 / 6
29:    original-tests/crypto-test/crypto/part 1/genKeys:         6 / 6
30:    original-tests/crypto-test/crypto/part 1/rsaEncrypt:       4 / 4
31:    original-tests/crypto-test/crypto/part 1/rsaDecrypt:       4 / 4
32:    original-tests/crypto-test/crypto/part 2/alphaOrd:         3 / 3
33:    original-tests/crypto-test/crypto/part 2/alphaChr:         3 / 3
34:    original-tests/crypto-test/crypto/part 2/add:              3 / 3
35:    original-tests/crypto-test/crypto/part 2/subtract:         3 / 3
36:    original-tests/crypto-test/crypto/part 2/ecbEncrypt:       4 / 4
37:    original-tests/crypto-test/crypto/part 2/ecbDecrypt:       4 / 4
38:    original-tests/crypto-test/crypto/part 2/cbcEncrypt:       4 / 4
39:    original-tests/crypto-test/crypto/part 2/cbcDecrypt:       4 / 4
40:
41: Git Repo: git@gitlab.doc.ic.ac.uk:lab2526_autumn/haskellcrypto_kw1425.git
42: Commit ID: 0b916
43: Git Diff: https://gitlab.doc.ic.ac.uk/lab2526_autumn/haskellcrypto_kw1425/-/compare/master..0b916?from_project_id=129118
```

```

1: module Crypto ( gcd, smallestCoPrimeOf, computeCoeffs, inverse
2:                 , modPow, genKeys, rsaEncrypt, rsaDecrypt
3:                 , alphaOrd, alphaChr, (<+>), (<->)
4:                 , ecbEncrypt, ecbDecrypt, cbcEncrypt, cbcDecrypt ) where
5:
6: import Data.Char
7:
8: import Prelude hiding (gcd)
9:
10: {-
11: The advantage of symmetric encryption schemes like AES is that they are efficient
12: and we can encrypt data of arbitrary size. The problem is how to share the key.
13: The flaw of the RSA is that it is slow and we can only encrypt data of size lower
14: than the RSA modulus n, usually around 1024 bits (64 bits for this exercise!).
15:
16: We usually encrypt messages with a private encryption scheme like AES-256 with
17: a symmetric key k. The key k of fixed size 256 bits for example is then exchanged
18: via the asymmetric RSA.
19: -}
20:
21: -----
22: -- PART 1 : asymmetric encryption
23:
24: -- | Returns the greatest common divisor of its two arguments
25: gcd :: Int -> Int -> Int
26: gcd m 0 = m
27: gcd m n = gcd n (m `mod` n)
28:
29: -- | Returns the smallest integer that is coprime with phi
30: smallestCoPrimeOf :: Int -> Int
31: smallestCoPrimeOf m = h
32:   where
33:     (h:_) = [n | n <- [2..], gcd m n == 1]
34:
35: -- | Calculates (n^k mod m)
36: modPow :: Int -> Int -> Int -> Int
37: modPow n 0 m = 1 `mod` m
38: modPow n k m
39:   | even k = x
40:   | otherwise = (n * x) `mod` m
41:   where
42:     x = modPow ((n * n) `mod` m) (k `div` 2) m
43:
44: -- | Computes BÃ©zout coefficients for two integers
45: computeCoeffs :: Int -> Int -> (Int, Int)
46: computeCoeffs a 0 = (1, 0)
47: computeCoeffs a b = (v, (u - q * v)) -- the formula (v', (u' - qv'))
48:   where
49:     (q, r) = quotRem a b
50:     (u, v) = computeCoeffs b r
51:
52: -- | Inverse of n modulo ms
53: inverse :: Int -> Int -> Int
54: inverse n m
55:   | gcd n m == 1 = u `mod` m
56:   | otherwise = error "no inverse"
57:   where (u, _) = computeCoeffs n m
58:
59: {-|
60: Generates keys pairs (public, private) = ((e, n), (d, n))
61: given two "large" distinct primes, p and q
62: -}
63: genKeys :: Int -> Int -> ((Int, Int), (Int, Int))
64:
65: genKeys p q = ((e, n), (d, n))
66:   where

```

```

67:     n = p * q
68:     x = (p-1)*(q-1)
69:     e = smallestCoPrimeOf x
70:     d = inverse e x
71:
72: -- | This function performs RSA encryption
73: rsaEncrypt :: Int -- ^ value to encrypt
74:            -> (Int, Int) -- ^ public key
75:            -> Int
76: rsaEncrypt x (e, n) = modPow x e n
77:
78: -- | This function performs RSA decryption
79: rsaDecrypt :: Int -- ^ value to decrypt
80:            -> (Int, Int) -- ^ public key
81:            -> Int
82: rsaDecrypt x (d, n) = modPow x d n
83:
84: -----
85: -- PART 2 : symmetric encryption
86:
87: -- | Returns position of a letter in the alphabet
88: alphaOrd :: Char -> Int
89: alphaOrd x = ord (toLower x) - ord 'a'
90:
91: -- | Returns the nth letter
92: alphaChr :: Int -> Char
93: alphaChr x = chr (x + ord 'a')
94:
95: -- | "adds" two letters
96: (<+>) :: Char -> Char -> Char
97: (<+>) x y = alphaChr ((alphaOrd x + alphaOrd y) `mod` 26)
98:
99: -- | "subtracts" two letters
100: (<->) :: Char -> Char -> Char
101: (<->) x y = alphaChr ((alphaOrd x - alphaOrd y) `mod` 26)
102:
103: -- the next functions present
104: -- 2 modes of operation for block ciphers : ECB and CBC
105: -- based on a symmetric encryption function e/d such as "add"
106:
107: -- | ecb (electronic codebook) encryption with block size of a letter
108: ecbEncrypt :: Char -> [Char] -> [Char]
109: ecbEncrypt k m = [x <+> k | x <- m]
110:
111: -- | ecb (electronic codebook) decryption with a block size of a letter
112: ecbDecrypt :: Char -> [Char] -> [Char]
113: ecbDecrypt k c = [x <-> k | x <- c]
114:
115: -- | cbc (cipherblock chaining) encryption with block size of a letter
116: cbcEncrypt :: Char -- ^ public key
117:            -> Char -- ^ initialisation vector 'iv'
118:            -> [Char] -- ^ message 'm'
119:            -> [Char]
120: cbcEncrypt _ _ [] = []
121: cbcEncrypt k iv (h:ts) = c : cbcEncrypt k c ts
122:   where (c:_) = ecbEncrypt k [h <+> iv]
123:
124:
125: -- | cbc (cipherblock chaining) decryption with block size of a letter
126: cbcDecrypt :: Char -- ^ private key
127:            -> Char -- ^ initialisation vector 'iv'
128:            -> [Char] -- ^ message 'm'
129:            -> [Char]
130: cbcDecrypt _ _ [] = []
131: cbcDecrypt k iv (h:ts) = m : cbcDecrypt k h ts
132:   where

```

**Final Tests****Crypto.hs: 3/3****Katie Wan(kw1425):c1:21**

```
133:      (x:_) = ecbDecrypt k [h]  
134:      m = x <-> iv
```

```

1: module CodeBreaker (secretMsg, guessKey) where
2:
3: import Crypto (ecbDecrypt)
4: import Data.List
5:
6: -----
7: -- PART 3 (Extension) : cracking the code
8:
9: -- Shhh, it's a secret (this is a multi-line string, by the way)
10: secretMsg :: String
11: secretMsg = "cqnmmnyjacvncwxcolxvydcwrpbrcqnlxvydcnablrlwnlnmnyjacvncwjcervynarjul\
12:             \xuunpnuxwmxcqnmnyjacvncwqbjbxaxdwmorochj1jmnvrlbcjoojwmxcwncqxdbjw\
13:             \mbcdmncwbcfrqjxaxdwmbrrgqdwmanmbcdmhrwpdwmnapajmdjcnlxadabnbcfxqdwma\
14:             \nmyqmbcdmncwbjwmcfxqdwmanmvblbcdmncwbcqnmnyjacvncwcrbyanmxvrvjwcuh\
15:             \kjbnnrwcqngdgunhkdrumrwpfqr1qrbcqjanbfrcqcnvjcqbmnyjacvncwqxfnen\
16:             \ajubxqjbbbyj1nrwcqnrfruurjvynwnwhujkxajcxahjwmrwcqjnaxwjdcr1bjwmlq\
17:             \nvrljwnwprwnnarwpgncwbrxcwqnmnyjacvncwacjwbtbqrpquhrwdwrenabrchajw\
18:             \trwpcjkunb"
19:
20: -- (Some of) the most common letter sequences of various sizes in English
21: enFreqSingle, enFreqBi, enFreqTri :: [String]
22: enFreqSingle = ["e", "t", "a", "o", "i", "n", "s", "h", "r"]
23: enFreqBi      = ["th", "he", "in", "er", "an", "re", "on", "at", "en"]
24: enFreqTri     = ["the", "and", "ing", "ent", "ion", "her", "for"]
25:
26: -- m@(h:ts) assigns the entire list (h:ts) into a variable named m
27:
28: ngrams :: Int -> [a] -> [[a]]
29: ngrams n m
30:     | (length m) < n = []
31:     | otherwise = take n m : ngrams n ts
32:   where (_,ts) = m
33:
34: descSort :: [(w, Int)] -> [(w, Int)]
35: descSort x = reverse (sortOn snd x)
36:
37: freqs :: Int -> String -> [(String, Int)]
38: freqs n m = descSort frequencies
39:   where
40:     groups = group (sort (ngrams n m))
41:     frequencies = [(h, length g) | g@(h:ts) <- groups]
42:
43: commonPrefixLength :: [String] -> [String] -> Int
44: commonPrefixLength (x:xs) (y:ys)
45:     | x == y = 1 + commonPrefixLength xs ys
46:     | otherwise = 0
47:
48: freqNgrams :: Int -> String -> [String]
49: freqNgrams n m = fst (unzip (freqs n m))
50:
51: calcScore :: String -> Int
52: calcScore m = (commonPrefixLength single enFreqSingle) + (commonPrefixLength bi enFreqBi) + (commonPrefixLength tri enFreqTri)
53:   where
54:     single = freqNgrams 1 m
55:     bi = freqNgrams 2 m
56:     tri = freqNgrams 3 m
57:
58: guessKey :: String -> [(Char, Int)]
59: guessKey c = descSort [(key, calcScore (ecbDecrypt key c)) | key <- ['a'..'z']]
60:
61: {-
62: the key is 'j'
63: the secret message is
64: thedepartmentofcomputingisthecomputersciencedepartmentatimperialcollegelondononthed/
65: epartmenthasaroundfiftyacademicstaffandonethousandstudentswitharoundsixhundredstudyingund

```

```
ergraduatecoursestwowhundredphdstudentsandtwohundredmcsstudentsthedepartmentispredominantl
ybasedinthehuxleybuildingwhichitshareswiththematshdepartmenthoweveralsohasspaceinthewilli
ampennylaboratoryandintheaeronauticsandchemicalengineeringextensionthedepartmentrankshig
hlyinuniversityrankingtables
65: -}
```

## Final Tests

testResults.txt: 1/6

Katie Wan(kw1425):c1:21

```
1: ----- Test Output -----
2: copying crypto.cabal from skeleton
3: copying cabal.project from skeleton
4: Config file path source is default config file.
5: Config file not found: /root/.config/cabal/config
6: Writing default configuration to /root/.config/cabal/config
7: Resolving dependencies...
8: Build profile: -w ghc-9.6.7 -O1
9: In order, the following will be built (use -v for more details):
10: - call-stack-0.4.0 (lib) (requires build)
11: - colour-2.3.6 (lib) (requires build)
12: - crypto-0.1.0.0 (lib) (first run)
13: - prettyprinter-1.7.1 (lib) (requires build)
14: - splitmix-0.1.3.1 (lib) (requires build)
15: - tagged-0.8.9 (lib) (requires build)
16: - ansi-terminal-types-1.1.3 (lib) (requires build)
17: - random-1.3.1 (lib) (requires build)
18: - ansi-terminal-1.1.3 (lib) (requires build)
19: - QuickCheck-2.16.0.0 (lib) (requires build)
20: - prettyprinter-ansi-terminal-1.1.3 (lib) (requires build)
21: - optparse-applicative-0.19.0.0 (lib) (requires build)
22: - tasty-1.5.3 (lib) (requires build)
23: - tasty-quickcheck-0.11.1 (lib) (requires build)
24: - tasty-hunit-0.10.2 (lib) (requires build)
25: - crypto-0.1.0.0 (test:crypto-properties) (first run)
26: - crypto-0.1.0.0 (test:crypto-test) (first run)
27: Starting    call-stack-0.4.0 (lib)
28: Configuring library for crypto-0.1.0.0...
29: Starting    colour-2.3.6 (lib)
30: Starting    prettyprinter-1.7.1 (lib)
31: Building    call-stack-0.4.0 (lib)
32: Building    colour-2.3.6 (lib)
33: Building    prettyprinter-1.7.1 (lib)
34: Preprocessing library for crypto-0.1.0.0...
35: Building library for crypto-0.1.0.0...
36: [1 of 2] Compiling Crypto      ( src/Crypto.hs, /tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/build/Crypto.o, /
/tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/build/Crypto.dyn_o )
37: [2 of 2] Compiling CodeBreaker  ( src/CodeBreaker.hs, /tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/build/CodeBreaker.o, /
/tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/build/CodeBreaker.dyn_o )
38: Installing  call-stack-0.4.0 (lib)
39: Starting    splitmix-0.1.3.1 (lib)
40: Completed   call-stack-0.4.0 (lib)
41: Starting    tagged-0.8.9 (lib)
42: Building    splitmix-0.1.3.1 (lib)
43: Building    tagged-0.8.9 (lib)
44: Installing  colour-2.3.6 (lib)
45: Completed   colour-2.3.6 (lib)
46: Starting    ansi-terminal-types-1.1.3 (lib)
47: Installing  splitmix-0.1.3.1 (lib)
48: Completed   splitmix-0.1.3.1 (lib)
49: Starting    random-1.3.1 (lib)
50: Installing  tagged-0.8.9 (lib)
51: Building    ansi-terminal-types-1.1.3 (lib)
52: Completed   tagged-0.8.9 (lib)
53: Building    random-1.3.1 (lib)
54: Installing  ansi-terminal-types-1.1.3 (lib)
55: Completed   ansi-terminal-types-1.1.3 (lib)
56: Starting    ansi-terminal-1.1.3 (lib)
57: Building    ansi-terminal-1.1.3 (lib)
58: Installing  ansi-terminal-1.1.3 (lib)
59: Completed   ansi-terminal-1.1.3 (lib)
```

## Final Tests

testResults.txt: 2/6

Katie Wan(kw1425):c1:21

```
60: Installing prettyprinter-1.7.1 (lib)
61: Completed prettyprinter-1.7.1 (lib)
62: Starting prettyprinter-ansi-terminal-1.1.3 (lib)
63: Building prettyprinter-ansi-terminal-1.1.3 (lib)
64: Installing prettyprinter-ansi-terminal-1.1.3 (lib)
65: Completed prettyprinter-ansi-terminal-1.1.3 (lib)
66: Starting optparse-applicative-0.19.0.0 (lib)
67: Building optparse-applicative-0.19.0.0 (lib)
68: Installing random-1.3.1 (lib)
69: Completed random-1.3.1 (lib)
70: Starting QuickCheck-2.16.0.0 (lib)
71: Installing optparse-applicative-0.19.0.0 (lib)
72: Completed optparse-applicative-0.19.0.0 (lib)
73: Starting tasty-1.5.3 (lib)
74: Building QuickCheck-2.16.0.0 (lib)
75: Building tasty-1.5.3 (lib)
76: Installing tasty-1.5.3 (lib)
77: Completed tasty-1.5.3 (lib)
78: Starting tasty-hunit-0.10.2 (lib)
79: Building tasty-hunit-0.10.2 (lib)
80: Installing tasty-hunit-0.10.2 (lib)
81: Completed tasty-hunit-0.10.2 (lib)
82: Configuring test suite 'crypto-test' for crypto-0.1.0.0...
83: Preprocessing test suite 'crypto-test' for crypto-0.1.0.0...
84: Building test suite 'crypto-test' for crypto-0.1.0.0...
85: [1 of 1] Compiling Main (test/Tests.hs, /tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/t/crypto-test/build/crypto-test/crypto-test-tmp/Main.o )
86: [2 of 2] Linking /tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/t/crypto-test/build/crypto-test/crypto-test
87: Installing QuickCheck-2.16.0.0 (lib)
88: Completed QuickCheck-2.16.0.0 (lib)
89: Starting tasty-quickcheck-0.11.1 (lib)
90: Building tasty-quickcheck-0.11.1 (lib)
91: Installing tasty-quickcheck-0.11.1 (lib)
92: Completed tasty-quickcheck-0.11.1 (lib)
93: Configuring test suite 'crypto-properties' for crypto-0.1.0.0...
94: Preprocessing test suite 'crypto-properties' for crypto-0.1.0.0...
95: Building test suite 'crypto-properties' for crypto-0.1.0.0...
96: [1 of 1] Compiling Main (test/Props.hs, /tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/t/crypto-properties/build/crypto-properties/crypto-properties-tmp/Main.o )
97: [2 of 2] Linking /tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/t/crypto-properties/build/crypto-properties/crypto-properties
98: Build profile: -w ghc-9.6.7 -O1
99: In order, the following will be built (use -v for more details):
100: - crypto-0.1.0.0 (test:crypto-test) (ephemeral targets)
101: Preprocessing test suite 'crypto-test' for crypto-0.1.0.0...
102: Building test suite 'crypto-test' for crypto-0.1.0.0...
103: Running 1 test suites...
104: Test suite crypto-test: RUNNING...
105: crypto
106:   part 1
107:     gcd
108:       #1: OK
109:       #2: OK
110:       #3: OK
111:       #4: OK
112:       #5: OK
113:       #6: OK
114:       #7: OK
115:       #8: OK
116:     phi
117:       #1: OK
118:       #2: OK
```

## Final Tests

testResults.txt: 3/6

Katie Wan(kw1425):c1:21

```
119:      #3: OK
120:      #4: OK
121:      #5: OK
122:      #6: OK
123:      #7: OK
124:      #8: OK
125:  modPow
126:      #1: OK
127:      #2: OK
128:      #3: OK
129:      #4: OK
130:      #5: OK
131:      #6: OK
132:      #7: OK
133:      #8: OK
134:      #9: OK
135:     #10: OK
136:     #11: OK
137:  computeCofeffs
138:      #1: OK
139:      #2: OK
140:      #3: OK
141:      #4: OK
142:      #5: OK
143:      #6: OK
144:  inverse
145:      #1: OK
146:      #2: OK
147:      #3: OK
148:      #4: OK
149:      #5: OK
150:      #6: OK
151:      #7: OK
152:  smallestCoPrimeOf
153:      #1: OK
154:      #2: OK
155:      #3: OK
156:      #4: OK
157:      #5: OK
158:      #6: OK
159:  genKeys
160:      #1: OK
161:      #2: OK
162:      #3: OK
163:      #4: OK
164:      #5: OK
165:      #6: OK
166:  rsaEncrypt
167:      #1: OK
168:      #2: OK
169:      #3: OK
170:      #4: OK
171:  rsaDecrypt
172:      #1: OK
173:      #2: OK
174:      #3: OK
175:      #4: OK
176:  part 2
177:  alphaOrd
178:      #1: OK
179:      #2: OK
```



## Final Tests

testResults.txt: 4/6

Katie Wan(kw1425):c1:21

```
180:      #3: OK
181:      alphaChr
182:      #1: OK
183:      #2: OK
184:      #3: OK
185:      add
186:      #1: OK
187:      #2: OK
188:      #3: OK
189:      subtract
190:      #1: OK
191:      #2: OK
192:      #3: OK
193:      ecbEncrypt
194:      #1: OK
195:      #2: OK
196:      #3: OK
197:      #4: OK
198:      ecbDecrypt
199:      #1: OK
200:      #2: OK
201:      #3: OK
202:      #4: OK
203:      cbcEncrypt
204:      #1: OK
205:      #2: OK
206:      #3: OK
207:      #4: OK
208:      cbcDecrypt
209:      #1: OK
210:      #2: OK
211:      #3: OK
212:      #4: OK
213:
214: All 88 tests passed (0.01s)
215:
216: Test suite crypto-test: PASS
217: Test suite logged to:
218: /tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/t/crypto-test/test/crypto-0.1.0.0-crypto-test.log
219: 1 of 1 test suites (1 of 1 test cases) passed.
220: copying test from skeleton
221: Build profile: -w ghc-9.6.7 -O1
222: In order, the following will be built (use -v for more details):
223: - crypto-0.1.0.0 (test:crypto-test) (additional components to build)
224: Preprocessing test suite 'crypto-test' for crypto-0.1.0.0...
225: Building test suite 'crypto-test' for crypto-0.1.0.0...
226: Build profile: -w ghc-9.6.7 -O1
227: In order, the following will be built (use -v for more details):
228: - crypto-0.1.0.0 (test:crypto-test) (ephemeral targets)
229: Preprocessing test suite 'crypto-test' for crypto-0.1.0.0...
230: Building test suite 'crypto-test' for crypto-0.1.0.0...
231: Running 1 test suites...
232: Test suite crypto-test: RUNNING...
233: crypto
234:   part 1
235:     gcd
236:     #1: OK
237:     #2: OK
238:     #3: OK
239:     #4: OK
240:     #5: OK
```

**Final Tests**

testResults.txt: 5/6

Katie Wan(kw1425):c1:21

```
241:      #6: OK
242:      #7: OK
243:      #8: OK
244:  phi
245:      #1: OK
246:      #2: OK
247:      #3: OK
248:      #4: OK
249:      #5: OK
250:      #6: OK
251:      #7: OK
252:      #8: OK
253:  modPow
254:      #1: OK
255:      #2: OK
256:      #3: OK
257:      #4: OK
258:      #5: OK
259:      #6: OK
260:      #7: OK
261:      #8: OK
262:      #9: OK
263:     #10: OK
264:     #11: OK
265:  computeCoeffs
266:      #1: OK
267:      #2: OK
268:      #3: OK
269:      #4: OK
270:      #5: OK
271:      #6: OK
272:  inverse
273:      #1: OK
274:      #2: OK
275:      #3: OK
276:      #4: OK
277:      #5: OK
278:      #6: OK
279:      #7: OK
280:  smallestCoPrimeOf
281:      #1: OK
282:      #2: OK
283:      #3: OK
284:      #4: OK
285:      #5: OK
286:      #6: OK
287:  genKeys
288:      #1: OK
289:      #2: OK
290:      #3: OK
291:      #4: OK
292:      #5: OK
293:      #6: OK
294:  rsaEncrypt
295:      #1: OK
296:      #2: OK
297:      #3: OK
298:      #4: OK
299:  rsaDecrypt
300:      #1: OK
301:      #2: OK
```

## Final Tests

testResults.txt: 6/6

Katie Wan(kw1425):c1:21

```
302:      #3: OK
303:      #4: OK
304:    part 2
305:      alphaOrd
306:      #1: OK
307:      #2: OK
308:      #3: OK
309:      alphaChr
310:      #1: OK
311:      #2: OK
312:      #3: OK
313:    add
314:      #1: OK
315:      #2: OK
316:      #3: OK
317:    subtract
318:      #1: OK
319:      #2: OK
320:      #3: OK
321:    ecbEncrypt
322:      #1: OK
323:      #2: OK
324:      #3: OK
325:      #4: OK
326:    ecbDecrypt
327:      #1: OK
328:      #2: OK
329:      #3: OK
330:      #4: OK
331:    cbcEncrypt
332:      #1: OK
333:      #2: OK
334:      #3: OK
335:      #4: OK
336:    cbcDecrypt
337:      #1: OK
338:      #2: OK
339:      #3: OK
340:      #4: OK
341:
342: All 88 tests passed (0.02s)
343:
344: Test suite crypto-test: PASS
345: Test suite logged to:
346: /tmp/d20251009-34-thyswl/dist-newstyle/build/x86_64-linux/ghc-9.6.7/crypto-0.1.0.0/t/crypto-test/test/crypto-0.1.0.0-crypto-test.log
347: 1 of 1 test suites (1 of 1 test cases) passed.
348:
349: ----- Test Errors -----
350: Warning: The package list for 'hackage.haskell.org' is 15 days old.
351: Run 'cabal update' to get the latest list of available packages.
```